



Project Report

Group 3

Medicine and Vitamin Logger

Submission Date: 9th August, 2026

NAME	ID	CONTRIBUTIONS	
Entisar Mahmud Emon	2321736642	Service Package	
Niladry Roy Chowdhury	2533918042	UI Package	
Sameeha Farheen Rashid	2534540642	Model Package	
COURSE	CSE215	SECTION	11
FACULTY	Dr Shamim Al Mamun (SAM3)	SEMESTER	Summer 2026

1. Introduction

Depending on individual conditions, a patient may have to take only one prescribed medicine for one week, or 5 different ones at different frequencies month-long. This difference in prescription leads to difficulty in maintaining adherence to the medications, especially among elderly patients who usually have many different medicines prescribed. This lack of adherence and forgetfulness might lead to dangerous consequences such as anti-biotic resistance.

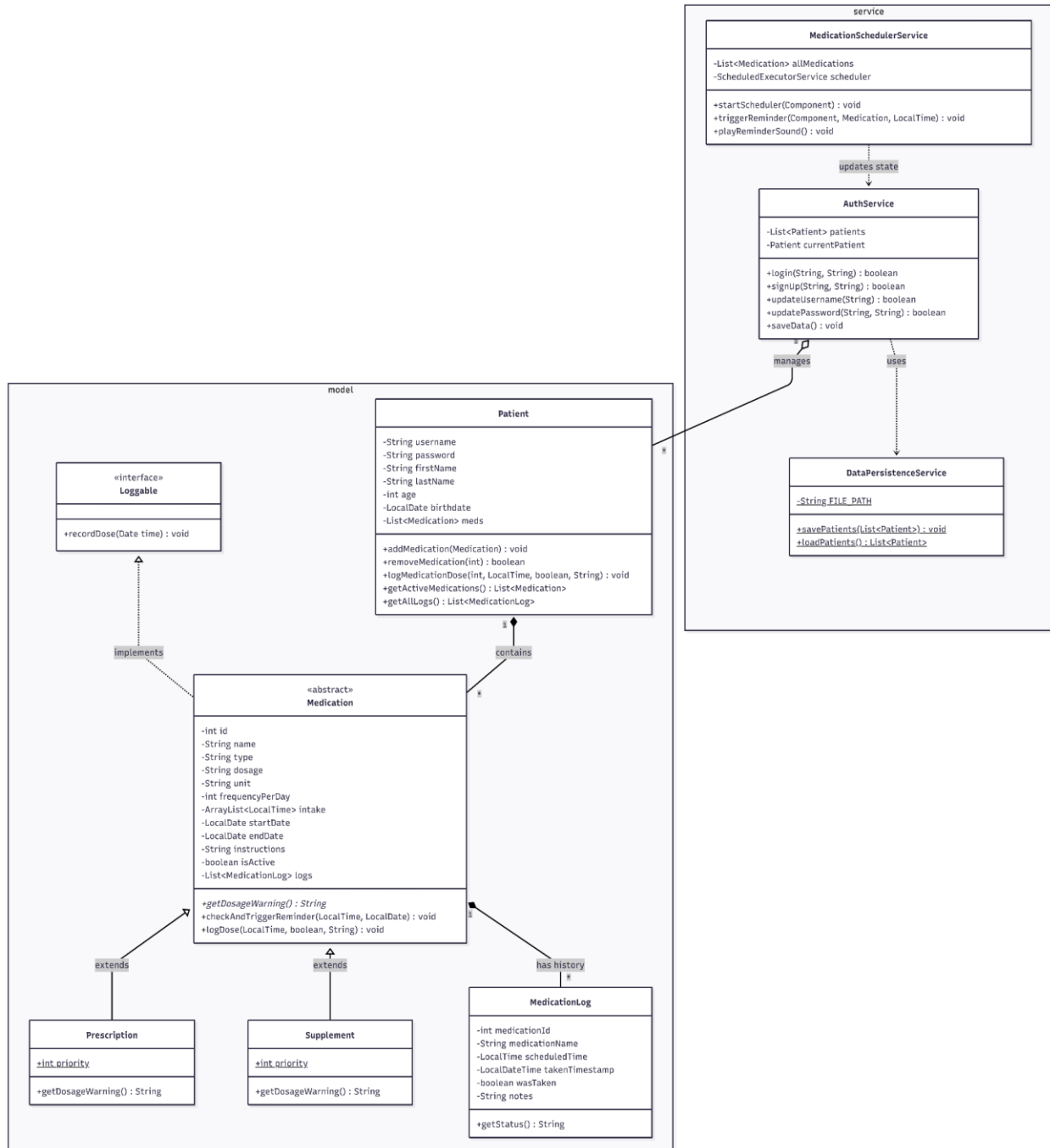
To prevent such a case, we developed the Medicine and Vitamin Logger, which contains Object-Oriented Programming (OOP) using Java, with Graphic User Interface (GUI) developed using JavaSwing as its base. The project also contains multithreading, lambda methods and file handling.

2. System Requirements and Specifications

2.1 Development

- **Language:** Java 17+
- **GUI Toolkit:** Java Swing
- **Concurrency:** *java.util.concurrent (ScheduledExecutorService)*

3. System Architecture and Design



3.1 Model Package

- **Loggable Interface:** The interface declares method *recordDose(LocalDate time)* for recording dose timestamps, when the specific checkboxes are selected.
- **Medication (Abstract Base Class):** The class implements the interfaces *Serializable* and *Loggable* and encapsulates certain attributes (such as ID, name, type, dosage, frequency, intake times, start and end dates, and instructions). It also declares abstract method *getDosageWarning()*.
- **Prescription & Supplement Subclasses:** The two classes inherited from *Medication*, within which, the method *getDosageWarning()* is overridden to provide classification-specific warnings.
- **Patient Class:** The class serves as the user model holding personal information, credentials, and a list of assigned medications, all of which are encapsulated.
- **MedicationLog Class:** It represents individual historical dose records, capturing timestamps, scheduled intake times, taken status, and additional notes (*Taken, Late, Missed*).

3.2 Service Package

- **AuthService:** This class controls several user account related actions such as account authentication, new user registration, credential modifications.
- **DataPersistenceService:** It manages file I/O operations. Reads and writes *List<Patient>* state to disk (data/patients.dat) using Java Serialization (*ObjectInputStream / ObjectOutputStream*).
- **MedicationSchedulerService:** Runs a *ScheduledExecutorService* background daemon checking every second to trigger reminders and play chime sounds when dose times match the current time.

3.3 UI Package

- **AuthFrame:** Contains LoginPanel and SignupPanel within a CardLayout container.
- **MainFrame:** Primary application hub containing a tabbed interface with system tray minimization, and auto-refresh timers.
- **DashboardPanel:** Highlights greetings, date-time display, summary cards (Doses completed, Next reminder), and time-categorized dose check-off rows (Morning, Noon, After Noon, Night).
- **MedicationManagerPanel:** Provides table viewing, searching, type filtering, detailed side-panel views, and full Create, Read, Update and Delete (CRUD) operations.
- **HistoryPanel:** Presents comprehensive dose log tables with status-color rendering (*Taken*, *Late*, *Missed*) and overall progress bars.
- **Theme Utility Class:** Global color palette manager facilitating real-time theme swapping.

All of the private attributes contain getter and setter methods in their respective classes.

4. Objectives

4.1 Different Users

Different patients will have different prescriptions and supplements, hence a log-in interface was created so that each user could be kept separate, along with their medications and schedules.

Each patient is separated using Patient objects with encapsulated attributes *username*, *password*, *name*, *age* and *birthdate*. *AuthService* class of *service* package maintains a list of Patient objects, which it uses to authenticate users when they are logging in. After that, unless the user logs out after their session, the user stays logged in since their credentials are saved in the pointer object *currentPatient*, which is then exposed to the UI via *authService.getCurrentPatient()*.

4.2 Dashboard

The dashboard acts as the central monitoring point, where the current time is displayed, along with the scheduled medicines, with the times being separated into four parts— Morning, Noon, Afternoon and Night. Here each of the medicines is displayed under their respective timing section with a checkbox beside with the help of *JCheckBox*, with the box being selected with the help of *addActionListener()* which is a lambda function.

The medications and its schedule is fetched using *getTodayDose()*, to display the medications under one of the four timing classifications. *createSummaryCards()* is used to display the adherence to the daily medications and the next reminder.

4.3 Medications

This panel allows for CRUD operations on the medication list, where `handleAddMedication()`, `handleEditMedication` and `handleRemoveMedication` are used where the Patient object is updated using `addMedication()` or `removeMedication()`. The medications are read using `refreshTableData()`, with the help of `getMedications()` getter, and dose timings are formatted with the help of `formatIntakeTime()`.

The user can also inspect each of the medications and its details by selecting the row, which triggers `updateDetailsPanel()`, which populates a side panel where intake instructions, active status, and dosage advice are displayed with the help of `getDosageWarning()`.

4.4 History

The history panel shows all the past logs of medications, with the help of `refreshHistory()`, collecting all the logs using `getLogs()` and populates a `DefaultTableModel` with intake dates, medication timing, notes and scheduled timing. There is also a `createAdherencePanel` where the adherence is shown on a progress bar, along with showing the number of dates for the medication and frequency, from which the number of medication is calculated. The adherence is calculated using $(\text{Taken Doses} / \text{Total Doses}) * 100$.

The method `applyFilters()` applies a `TableRowSorter` to filter intake records by outcome (All, Taken, Late, Missed), with colour cues from using `StatusRenderer` (`DefaultTableCellRenderer`).

4.5 Reminder

Many patients forget to take their medicines on time. To prevent such from occurring, an alert feature was developed here, using a `checkAndTriggerReminder()` method in the *model* package to check the current time against the listed times for the active medications. To run the method continuously, a separate service class was created known as the *MedicationSchedulerService* of the service package. Here, multithreading is used to continuously run the `triggerReminder()` method, which runs the aforementioned method every 1000ms.

5. Conclusion

The Medicine and Vitamin Logger project demonstrates the major concepts of OOP, with its graphic interface made using Java Swing. The project takes a major issue such as the scheduling and punctuality of medication and turns it into a simplified dashboard, with information separated according to the users, and customizable scheduling.

6. References

1. Medisafe Medication Management by MediSafe Inc.
2. Assistance by Gemini for Graphical User Interface (GUI)
3. www.drugs.com